

Pontifícia Universidade Católica do Paraná

Disciplina: Técnicas de Machine Learning

Somativa 2

Aluno: Vanderlei Sutil de Córdova

Mantendo as importações da somativa 1.

```
In [1]: import pandas as pd # biblioteca usada para manipulação de dataframes
import numpy as np # manipulação de tabelas
import matplotlib.cm as mcm # biblioteca para mostrar gráficos (especificamente um
import matplotlib.pyplot as plt # biblioteca para mostrar gráficos (especificament
import seaborn as sns # outra biblioteca para mostrar gráficos (ela é específica

from ydata_profiling import ProfileReport # Biblioteca para análise exploratória
#from sklearn.preprocessing import LabelEncoder # Biblioteca para normalização d
from sklearn.feature_selection import *
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import * # importando todas as funções específicas pa
from sklearn.cluster import * # importando todas as funções específicas para o a

from sklearn.model_selection import train_test_split, RandomizedSearchCV, GridSe
from sklearn.ensemble import RandomForestClassifier # Biblioteca para treinament

#Bibliotecas para somativa2
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import OneHotEncoder
from sklearn.pipeline import Pipeline # Biblioteca para criação de pipelines.
from sklearn import set_config # utilizado para mostrar os passos do pipeline de
set_config(display='diagram') # forçando para que os passos do pipeline sejam mo
from sklearn.metrics import * # importando todas as funções de métricas do sciki
```

[Upgrade to ydata-sdk](#)

Improve your data and profiling with ydata-sdk, featuring data quality scoring, redundancy detection, outlier identification, text validation, and synthetic data generation.

```
In [2]: %matplotlib inline
```

Etapa de carregamento dos dados apenas.

```
In [3]: # Carregamento do dataset
df = pd.read_excel('csgo_round_snapshots.xlsx', index_col=False)
df.head()
```

Out[3]:

	time_left	ct_score	t_score	map	ct_health	t_health	ct_money	t_money	ct_hel
0	175.00	0	0	de_dust2	500	500	4000	4000	
1	156.03	0	0	de_dust2	500	500	600	650	
2	96.03	0	0	de_dust2	391	400	750	500	
3	76.03	0	0	de_dust2	391	400	750	500	
4	174.97	1	0	de_dust2	500	500	18350	10750	

5 rows × 94 columns

In [4]: *# Além de remover duplicatas vou remover as colunas referentes as granadas.*
`df = df.drop_duplicates().drop(columns=df.columns[df.columns.str.contains('grana')])`
`df`

Out[4]:

	time_left	ct_score	t_score	map	ct_health	t_health	ct_money	t_money
0	175.00	0	0	de_dust2	500	500	4000	4000
1	156.03	0	0	de_dust2	500	500	600	650
2	96.03	0	0	de_dust2	391	400	750	500
3	76.03	0	0	de_dust2	391	400	750	500
4	174.97	1	0	de_dust2	500	500	18350	10750
...	...	...	...	...	...	...	...	...
122405	15.41	11	14	de_train	200	242	100	5950
122406	174.93	11	15	de_train	500	500	11500	23900
122407	114.93	11	15	de_train	500	500	1200	6700
122408	94.93	11	15	de_train	500	500	1200	6700
122409	74.93	11	15	de_train	375	479	1100	7000

117445 rows × 82 columns

Etapa de divisão do dataset entre treino e teste.

In [5]: `X_train, X_test, y_train, y_test = train_test_split(df.drop('round_winner', axis=1), df['round_winner'], test_size=0.25, random_state=42)`

Etapa de preparação dos dados

Existem divergências entre as técnicas adotadas na somativa 1 e nesta entrega. Após um melhor entendimento

do conteúdo eu aprimorei a preparação dos dados.

```
In [6]: # Aplicação do LabelEncoder para conversão dos dados do meu target (round_winner)
label_encoder = LabelEncoder() # Instanciação do LabelEncoder
y_train = label_encoder.fit_transform(y_train) # fit_transform apenas nos dados
y_test = label_encoder.transform(y_test) # transform no y_test
label_encoder.classes_ # Visualização da transformação dos dados (CT, T).

# Aplicação do OneHotEncoder para a coluna 'map' da feature.
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False) #Instanciação
coluna_map_train = X_train[['map']] # Seleção da coluna map na base de treino
coluna_map_test = X_test[['map']] # Seleção da coluna map na base de teste
map_encoded_train = ohe.fit_transform(coluna_map_train) # fit nos dados de trein
map_encoded_test = ohe.transform(coluna_map_test) # transform nos dados de teste

# Integrando as novas colunas aos DataFrames originais
# Crie DataFrames com as novas colunas codificadas, usando os nomes de features
map_df_train = pd.DataFrame(map_encoded_train, columns=ohe.get_feature_names_out())
map_df_test = pd.DataFrame(map_encoded_test, columns=ohe.get_feature_names_out())

# Junta os DataFrames: remove a coluna original 'map' e adiciona as novas colunas
X_train = pd.concat([X_train.drop('map', axis=1), map_df_train], axis=1)
X_test = pd.concat([X_test.drop('map', axis=1), map_df_test], axis=1)
```

Scaling para tratar as diferenças de escalas dos dados da feature

```
In [7]: scaler = StandardScaler() #Instanciação do Scaler
X_train_scaled = scaler.fit_transform(X_train) # fit_transform apenas nos dados
X_test_scaled = scaler.transform(X_test) # transform nos dados de teste

# Transformando em dataset novamente
X_train_scaled = pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X_test.columns)

# Visualizando as primeiras linhas dos dados escalonados
display("Cabeçalho do X_train_scaled:")
display(X_train_scaled.head())
```

'Cabeçalho do X_train_scaled:'

	time_left	ct_score	t_score	ct_health	t_health	ct_money	t_money	ct_helmets
0	-0.146309	-1.411093	-1.206043	-1.557119	-0.833579	-0.684094	-0.683503	-1.159112
1	1.227380	-0.160817	-0.170675	0.682942	0.716250	1.218838	-0.023136	1.572404
2	0.286875	0.881081	1.485915	0.682942	0.716250	0.441771	-0.769106	1.026101
3	1.507974	-0.160817	0.450546	0.682942	0.716250	-0.763567	-0.899549	-1.159112
4	-0.745222	1.714598	0.450546	0.682942	-0.727426	1.783978	-0.915854	1.572404

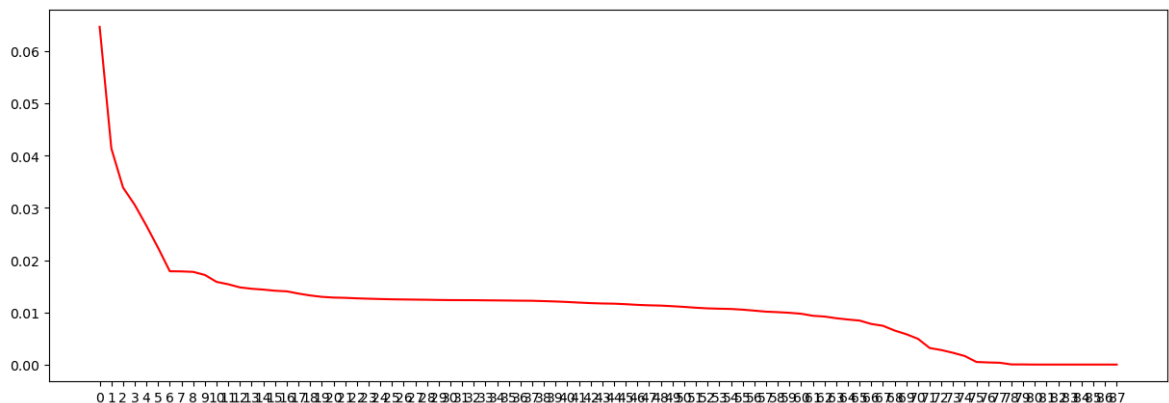
5 rows × 88 columns



Perfeito agora um boa parte do dataset está tratado, porém gostaria de utilizar o PCA para tratar a quantidade de colunas

```
In [8]: pca = PCA(n_components=X_train_scaled.shape[1]) # Instanciando o PCA com a quant
X_train_pca = pca.fit_transform(X_train_scaled)

#Técnica elbow para identificação do número ideal de componentes
plt.figure(figsize=(15, 5)) # criando um gráfico retangular para facilitar a vis
plt.plot(pca.explained_variance_ratio_, color='r') # colocando a porcentagem de
plt.xticks(np.arange(X_train_scaled.shape[1])) # mostrando todos os números no e
plt.show() # mostrando o gráfico final
```



A quantidade ideal de componentes é 6

```
In [9]: pca = PCA(n_components=6) # Instanciação do pca com 6 componentes
X_train_pca = pca.fit_transform(X_train_scaled) # fit_transform nos dados de tre
X_test_pca = pca.transform(X_test_scaled) # transform nos dados de teste

display(f'Número original de features: {X_train_scaled.shape[1]}')
display(f'Número de features após PCA: {pca.n_components_}')
```

'Número original de features: 88'

'Número de features após PCA: 6'

Etapa do treinamento do algoritmo com um algoritmo de aprendizagem supervisionada.

```
In [10]: rfc_model = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rfc_model.fit(X_train_pca, y_train) #fitando o modelo
display('Treinamento concluído')

display('Score') # Score do treinamento dos dados.
display(rfc_model.score(X_test_pca, y_test))

y_pred = rfc_model.predict(X_test_pca) # Realizando a predição no conjunto de te

df_comparacao = pd.DataFrame({'Valores Reais': y_test, 'Predições': y_pred}) # D
display('Comparação entre Valores Reais e Predições:')
display(df_comparacao.head(20))
```

'Treinamento concluído'

'Score'

0.7974252435120224

'Comparação entre Valores Reais e Predições:'

	Valores Reais	Predições
0	0	0
1	1	1
2	0	0
3	1	1
4	0	0
5	0	0
6	1	0
7	1	1
8	0	0
9	1	1
10	0	0
11	0	0
12	1	0
13	1	1
14	0	0
15	0	0
16	0	0
17	1	1
18	1	1
19	0	0

Aplicando o conceito de pipelines e métricas específicas.

```
In [11]: # Carregamento do dataset
df_pipeline = pd.read_excel('csgo_round_snapshots.xlsx', index_col=False)
# Além de remover duplicatas vou remover as colunas referentes as granadas.
df_pipeline = df_pipeline.drop_duplicates().drop(columns=df_pipeline.columns[df_
# Divisão do dataset
X_train, X_test, y_train, y_test = train_test_split(df_pipeline.drop('round_winn
df_pipeline['round_winner'],
test_size=0.25,
random_state=42)
```

Se faz necessário a conversão dos dados contido no target antes do encapsulamento do pipeline.

```
In [12]: # Aplicação do LabelEncoder para conversão dos dados do meu target (round_winner)
label_encoder = LabelEncoder() # Instanciação do LabelEncoder
y_train = label_encoder.fit_transform(y_train) # fit_transform apenas nos dados
y_test = label_encoder.transform(y_test) # transform no y_test
label_encoder.classes_ # Visualização da transformação dos dados (CT, T).
```

```
Out[12]: array(['CT', 'T'], dtype=object)
```

Pipeline

```
In [13]: pipe_1 = Pipeline([
    ('encoder', OneHotEncoder(handle_unknown='ignore', sparse_output=False)),
    ('scaler', StandardScaler()),
    ('pca', PCA(n_components=6)),
    ('modelo', RandomForestClassifier())
]).fit(X_train, y_train)

y_pred = pipe_1.predict(X_test)
display(f'Resultados de y_pred: {y_pred}')

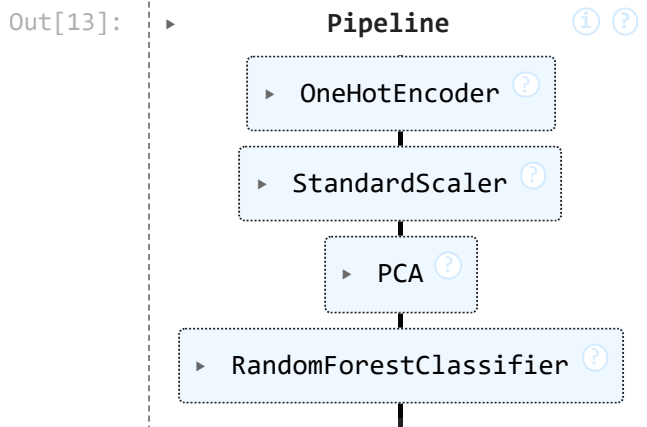
display(f'Score: {pipe_1.score(X_test, y_test)}')

display(f'Passos do pipeline')
pipe_1
```

```
'Resultados de y_pred: [0 1 1 ... 1 1 0]'
```

```
'Score: 0.774129827668415'
```

```
'Passos do pipeline'
```



Usando métricas de classificação

```
In [14]: display(f'Balanced accuracy: {balanced_accuracy_score(y_test, y_pred)}')
display(f'F1 Score: {f1_score(y_test, y_pred)}')
```

```
'Balanced accuracy: 0.774120031312123'
```

```
'F1 Score: 0.7771954579049922'
```

Conclusão

Por ser tratar de classes balanceadas (CT,T) a acurácia é uma boa escolha de métrica,

pois responde de forma clara a porcentagem de vezes que o modelo acerta o vencedor de cada round.

Já o score f1 nos permite tirar a prova como um complemento a acurácia garantindo que o modelo não está

apenas "chutando" uma classe ligeiramente marjoritária.

Com um acerto de aproximadamente 77% dos casos o modelo de fato apresenta resultados sólidos.

O modelo também mostrou não ter um "lado preferido" prevendo os resultados de CT e T com a mesma qualidade.

In []: